

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1 Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2 Learning Outcomes

Students will be able to:

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3 Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)

- Y : Feature Map

The output feature map size is

$$Output = \frac{N - F + 2P}{S} + 1$$

where

- N Input Size
- F Filter Size
- P Padding
- S Stride

4 Dataset Description

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

5 Experimental Procedure

5.1 Task 1

Load the CIFAR-10 dataset.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

5.2 Task 2

Implement a convolution layer.

Compare the following kernel sizes:

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record the corresponding feature map sizes.

5.3 Task 3

Study the effect of different hyperparameters.

Compare the following configurations:

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute the output dimensions for each configuration.

5.4 Task 4

Visualize the feature maps after the first convolution layer.

Display at least eight feature maps.

5.5 Task 5

Compare the following pooling techniques:

- Max Pooling
- Average Pooling

Compare their output size and classification accuracy.

5.6 Task 6

Construct the following CNN architecture:

Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax

Train the CNN using the following parameters:

- Optimizer: Adam
- Epochs: 20
- Batch Size: 32

5.7 Task 7

Evaluate the performance of the CNN using the following metrics:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

6 Results

The CNN was trained using the CIFAR-10 dataset. The training dataset contained 50,000 images, with 5,000 images belonging to each of the 10 classes. The images used for classification were of size $32 \times 32 \times 3$.

The CNN model used two convolution layers followed by max pooling, a flatten layer, a dense layer and an output layer. The model contained a total of 268,650 trainable parameters. The CNN was trained using the Adam optimizer for 20 epochs with a batch size of 32. The architecture followed the sequence:

Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax

The model was then evaluated on the CIFAR-10 test images to measure its classification performance.

6.1 CNN Model Performance

The final results obtained from the CNN are shown in Table 1.

Table 1: CNN Performance Results

Metric	Value
Training Accuracy	91.92%
Testing Accuracy	63.52%
Precision	64.07%
Recall	63.52%
F1-score	63.28%

From the results, the model was able to learn the training data well, with the training accuracy reaching about 91.92%. The testing accuracy was 63.52%. The precision, recall and F1-score were 64.07%, 63.52% and 63.28%, respectively.

6.2 Effect of Kernel Size

Three different kernel sizes were tested using 3×3 , 5×5 and 7×7 filters. The number of filters was kept at 8 while changing the kernel size.

Table 2: Feature Map Size for Different Kernel Sizes

Kernel Size	Output Height	Output Width	Feature Maps
3×3	30	30	8
5×5	28	28	8
7×7	26	26	8

The results show that increasing the kernel size reduces the output dimensions when valid padding is used. The 3×3 kernel produced a 30×30 feature map, while the 7×7 kernel reduced it further to 26×26 . The number of feature maps remained 8 in all three cases.

6.3 Effect of Stride and Padding

The effect of stride and padding was checked using the same input size and convolution filters.

Table 3: Effect of Stride and Padding

Configuration	Stride	Padding	Output Size	Feature Maps
Stride = 1	1	Valid	30×30	8
Stride = 2	2	Valid	15×15	8
Padding = Same	1	Same	32×32	8
Padding = Valid	1	Valid	30×30	8

It can be seen that increasing the stride from 1 to 2 reduced the feature-map size from 30×30 to 15×15 . When same padding was used, the output retained the original 32×32 spatial size.

6.4 Max Pooling and Average Pooling

Max pooling and average pooling were tested separately to see how they affected the classification performance.

Table 4: Comparison of Pooling Methods

Pooling Method	Output Size	Accuracy	Precision	Recall	F1-score
Max Pooling	$8 \times 8 \times 32$	68.80%	69.00%	68.80%	68.59%
Average Pooling	$8 \times 8 \times 32$	69.31%	69.20%	69.31%	69.18%

Inference:

Both max pooling and average pooling produced the same output size because the same 2×2 pooling window was used in both models. After the two pooling layers, the feature-map size was $8 \times 8 \times 32$. Average pooling gave slightly better results, with an accuracy of 69.31% compared to 68.80% for max pooling. The difference is small, but average pooling performed slightly better in this experiment.

Table 5: Effect of Increasing the Number of Filters

Filters	Testing Accuracy	Training Time (s)
16	66.85%	711.16
32	67.10%	1015.05
64	68.09%	2524.30

The accuracy increased gradually when more filters were used. However, the increase was small compared with the increase in training time. The 64-filter model gave the highest testing accuracy of 68.09%, but it also took 2524.30 seconds to train.

6.5 ReLU and Sigmoid Comparison

The effect of the activation function was tested by comparing ReLU and Sigmoid.

Table 6: Comparison of ReLU and Sigmoid

Activation	Testing Accuracy	Training Loss	Training Time (s)
ReLU	68.10%	0.6319	653.94
Sigmoid	55.26%	1.1422	710.61

ReLU performed better than Sigmoid for the CNN used in this experiment. The testing accuracy with ReLU was 68.10%, while Sigmoid gave only 55.26%. ReLU also reached a lower final training loss and took less time to train.

7 Plots with Inference

7.1 Sample Images

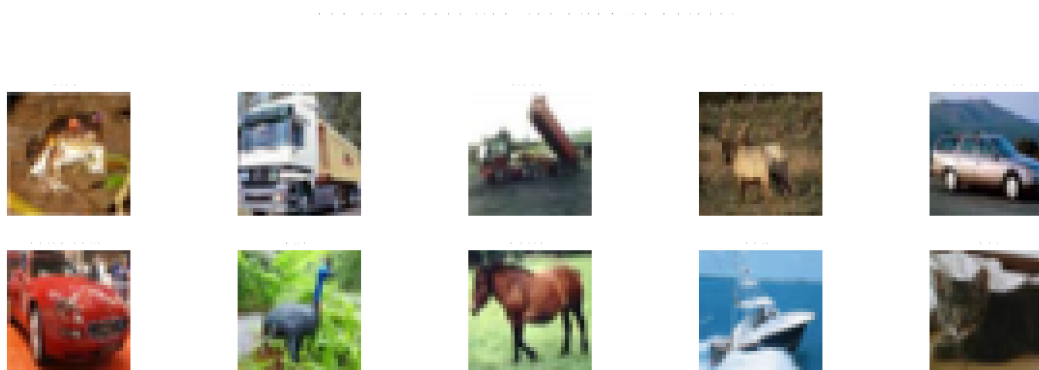


Figure 1: Sample images from the CIFAR-10 dataset

Inference:

The figure shows some sample images from the CIFAR-10 dataset. The images shown include Frog, Truck, Automobile, Bird and Horse, with Truck appearing twice. Since the images are only 32×32 pixels, the objects are quite small and have different colours, shapes and backgrounds. This makes it a little difficult for the model to identify the classes correctly.

7.2 Class Distribution

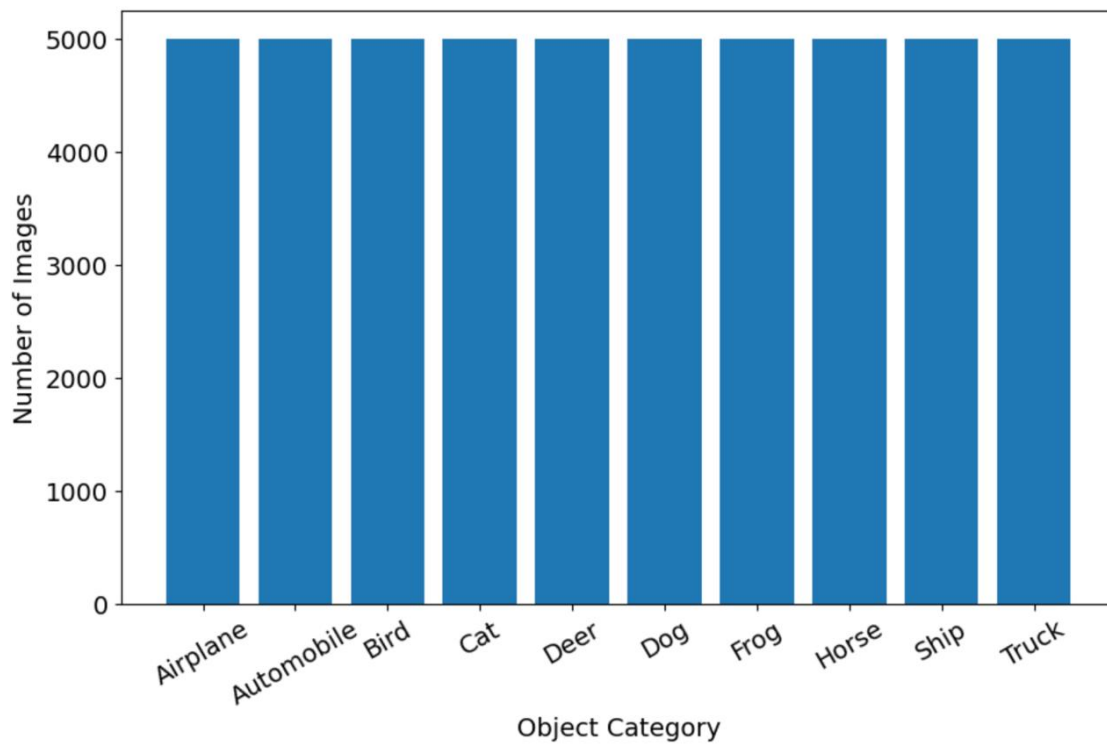


Figure 2: Class distribution in the CIFAR-10 training dataset

Inference:

The graph shows that all ten classes have exactly 5,000 training images each. The bars are therefore of equal height, showing that the training dataset is balanced and that no class has more training samples than the others.

7.3 Training and Validation Accuracy

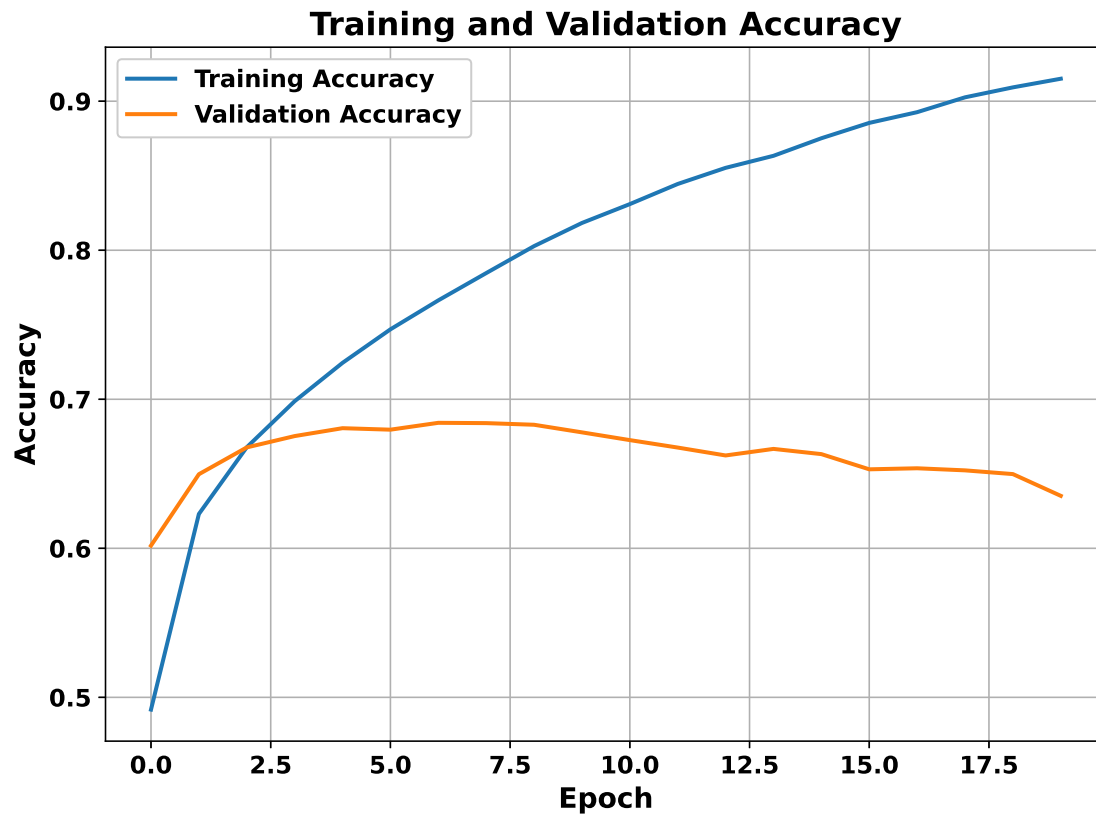


Figure 3: Training and validation accuracy over 20 epochs

Inference:

The training accuracy keeps increasing throughout the 20 epochs, starting at around 49% and reaching about 91.5% at the end. The validation accuracy increases quickly at first and reaches its highest value of around 68.4% near the seventh epoch. After that, it starts to drop and ends at 63.52%, while the training accuracy continues to increase. This shows that after a few epochs, the model is learning the training images too closely and is not improving much on the validation data.

7.4 Training and Validation Loss

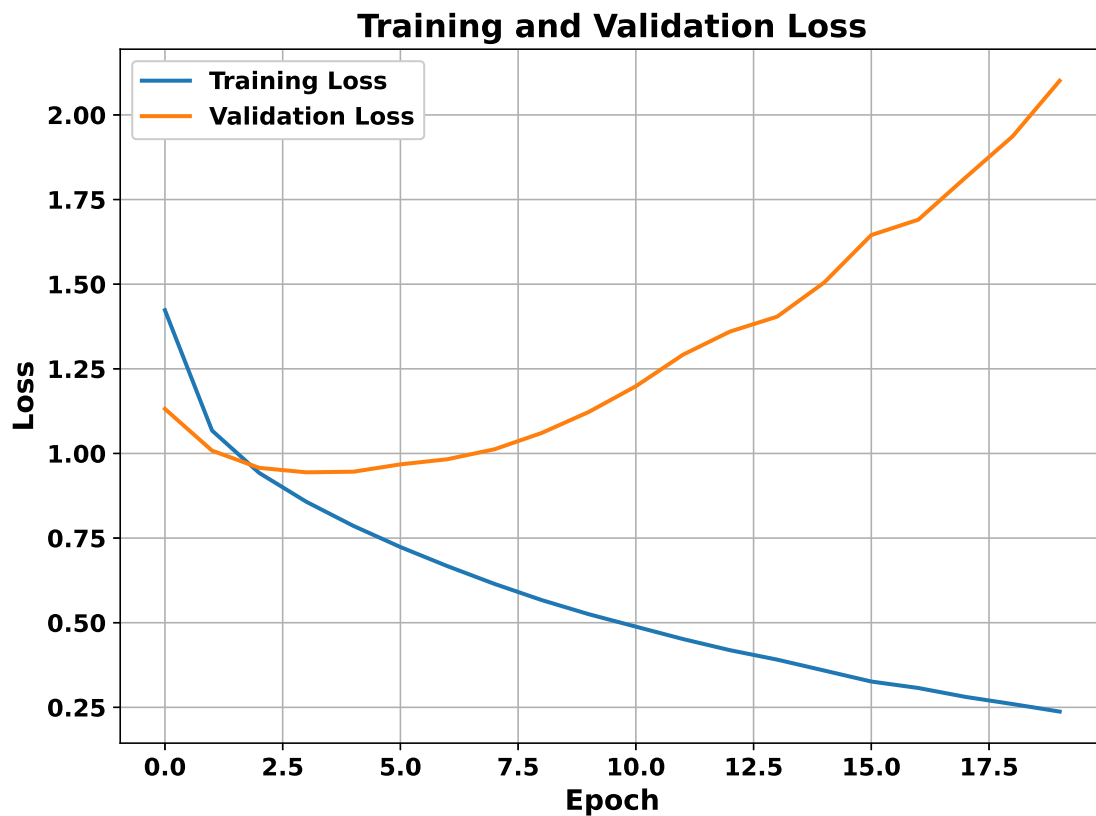


Figure 4: Training and validation loss over 20 epochs

Inference:

The training loss keeps going down throughout the 20 epochs, from around 1.42 in the first epoch to about 0.24 at the end. The validation loss also decreases in the beginning and reaches its lowest point at around the third epoch. After that, it starts increasing and reaches about 2.10 by the last epoch. This difference between the training and validation loss shows that the model starts overfitting the training data as the epochs increase.

7.5 Feature Map Visualization

Figure 7. Feature Maps after the First Convolution Layer

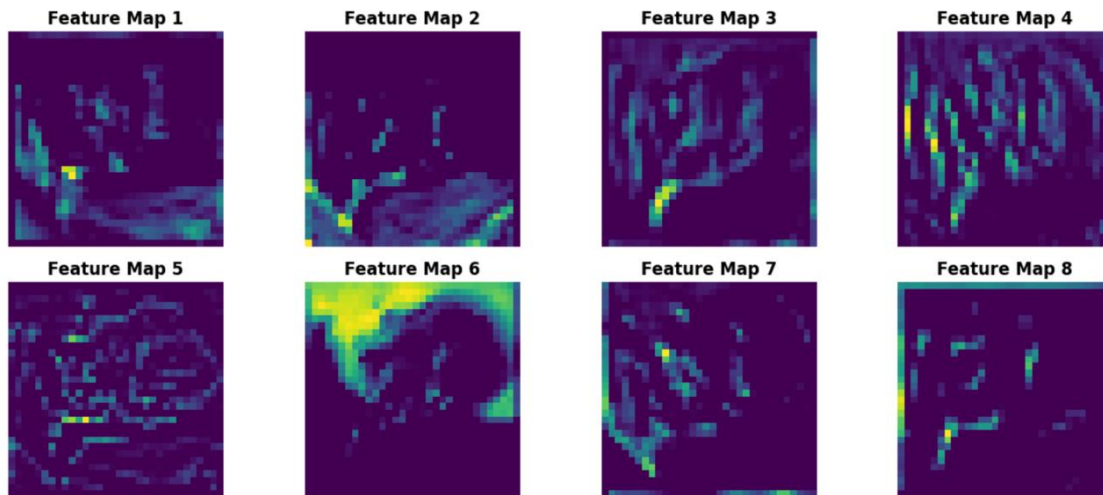


Figure 5: Feature maps obtained after the first convolution layer

Inference:

The feature maps from the first convolution layer show that each filter responds differently to the same input image. Some of the maps clearly highlight edges and brighter parts of the image, while others show different patterns and textures. This shows how the first convolution layer extracts different features from the image before sending them to the next layers.

7.6 Confusion Matrix

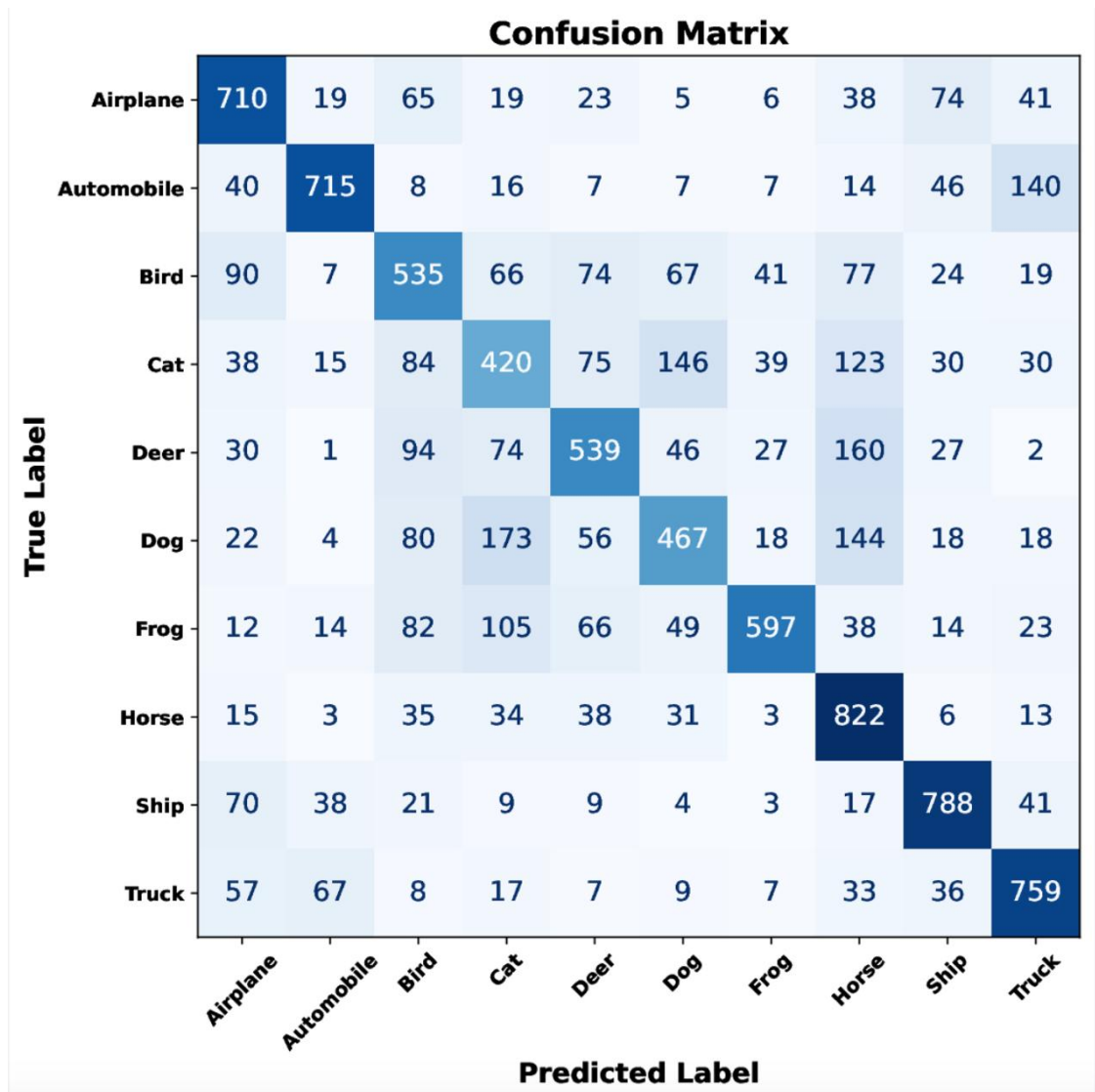


Figure 6: Confusion matrix obtained for the CNN

Inference:

The confusion matrix shows that the model classified the Horse class the best, with 822 out of 1,000 images predicted correctly. Ship and Truck also had good results, with 788 and 759 images classified correctly. The model had more difficulty with Cat and Dog, where only 420 and 467 images were correctly predicted. I also noticed that Cat and Dog were often confused with each other, with 146 Cat images predicted as Dog and 173 Dog images predicted as Cat.

7.7 Classification Report

The classification report gives the precision, recall and F1-score for each of the ten CIFAR-10 classes. It gives a more detailed view of how the model performed for each class.

Table 7: Classification Report for the CNN

Class	Precision	Recall	F1-score	Support
Airplane	0.6550	0.7100	0.6814	1000
Automobile	0.8097	0.7150	0.7594	1000
Bird	0.5287	0.5350	0.5318	1000
Cat	0.4502	0.4200	0.4346	1000
Deer	0.6029	0.5390	0.5692	1000
Dog	0.5620	0.4670	0.5101	1000
Frog	0.7981	0.5970	0.6831	1000
Horse	0.5607	0.8220	0.6667	1000
Ship	0.7413	0.7880	0.7639	1000
Truck	0.6989	0.7590	0.7277	1000
Accuracy		0.6352		10000
Macro Avg	0.6407	0.6352	0.6328	10000
Weighted Avg	0.6407	0.6352	0.6328	10000

Inference:

The classification report shows that the model performed best for the Automobile class in terms of F1-score, with a value of 0.7594. Horse had the highest recall at 0.8220, while Cat had the lowest F1-score of 0.4346. The overall accuracy was 63.52%, which shows that the model was able to classify the CIFAR-10 images reasonably well but still had difficulty with some classes.

7.8 Pooling Performance Comparison

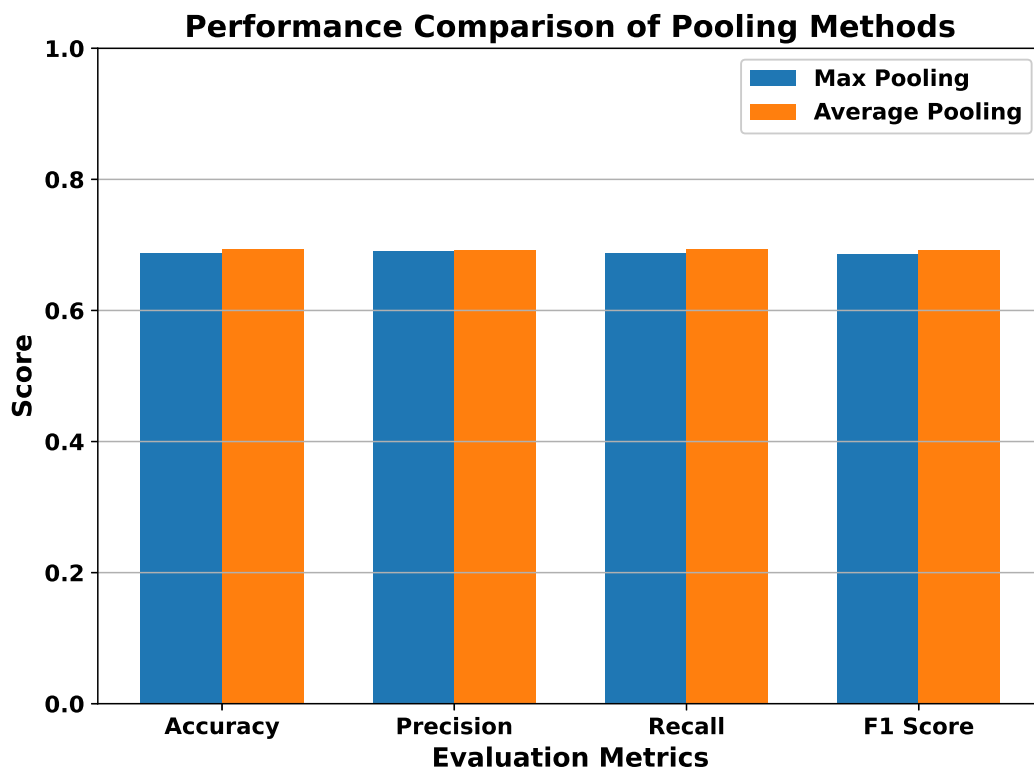


Figure 7: Performance comparison of max pooling and average pooling

Inference:

The plot shows that average pooling gave slightly better results than max pooling in this experiment. The accuracy was 69.31% with average pooling compared to 68.80% with max pooling. The F1-score was also a little higher for average pooling, at 69.18% compared to 68.59% for max pooling. Overall, the difference is small, but average pooling performed slightly better in our case.

7.9 Effect of Number of Filters

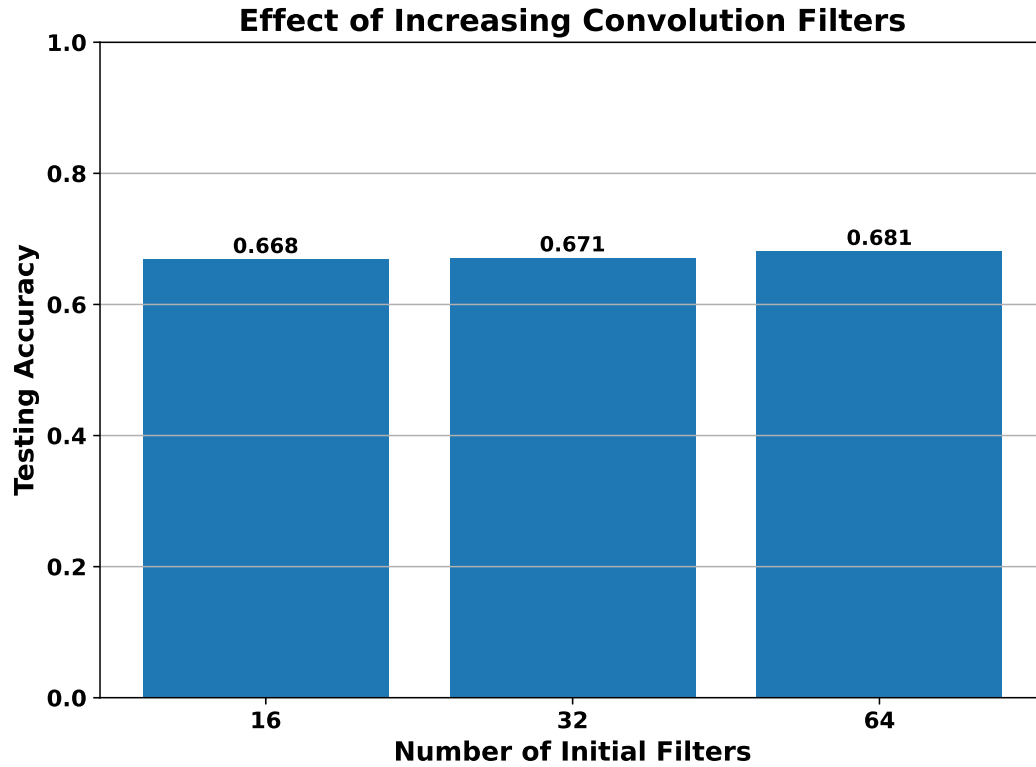


Figure 8: Testing accuracy for different numbers of convolution filters

Inference:

The graph shows that the testing accuracy increases only slightly as the number of filters is increased. It goes from 66.85% with 16 filters to 67.10% with 32 filters and reaches 68.09% with 64 filters. So, using more filters did improve the accuracy, but the improvement was quite small.

7.10 ReLU and Sigmoid Comparison

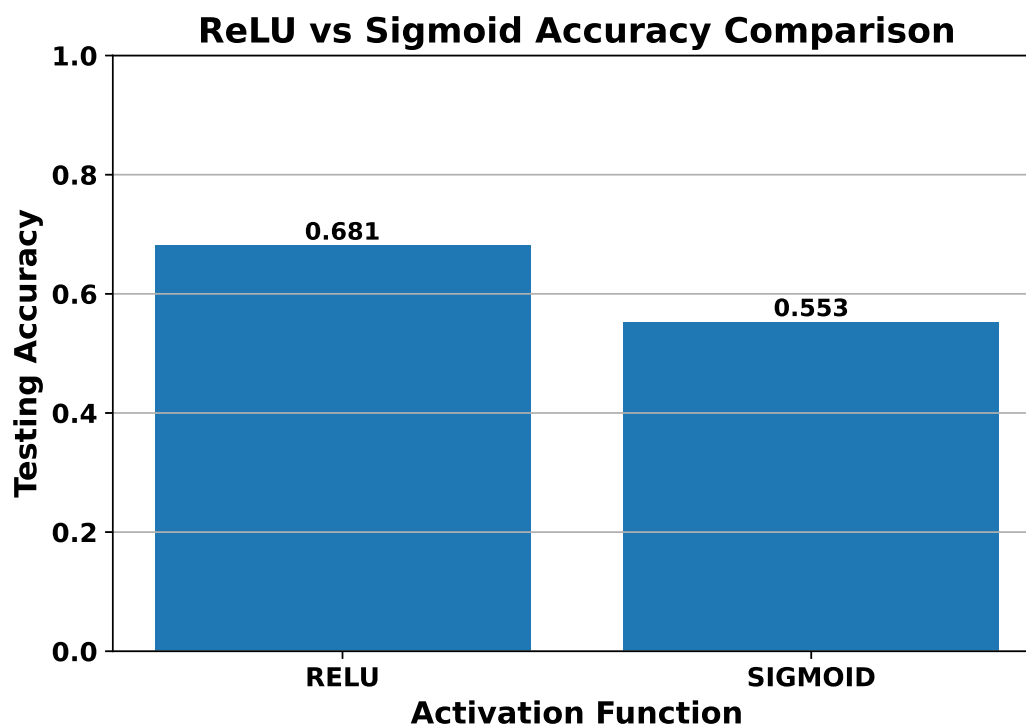


Figure 9: Testing accuracy comparison between ReLU and Sigmoid

Inference:

The graph shows a clear difference between the two activation functions. With ReLU, the model reached a testing accuracy of 68.10%, while Sigmoid reached only 55.26%. So, in our experiment, ReLU gave a better result by about 12.84 percentage points.

8 Discussion

8.1 Why is convolution preferred over fully connected layers for images?

In our experiment, the convolution layers first extracted useful features from the 32×32 CIFAR-10 images before the data was given to the dense layer. If we used a fully connected layer directly, every pixel would have to be connected to the neurons, which would increase the number of parameters. Convolution avoids this by looking at smaller parts of the image and using the same filters at different positions.

8.2 How does stride affect the feature map size?

The output-size experiment made the effect of stride clear. With a 3×3 kernel and valid padding, using stride 1 gave a 30×30 feature map, while stride 2 reduced it to 15×15 . So, when the stride is increased, the filter moves in bigger steps across the image and the output feature map becomes smaller.

8.3 What is the role of padding?

In our experiment, the padding setting directly affected the size of the output feature map. With the 3×3 kernel, valid padding gave a 30×30 feature map, while same padding kept the output size at 32×32 . So, same padding is useful when we want to keep the spatial size of the input unchanged.

8.4 Why is pooling used?

Pooling helps reduce the size of the feature maps before they are passed to the next layer. In our experiment, both max pooling and average pooling gave similar results, but average pooling performed slightly better. It achieved an accuracy of 69.31%, compared to 68.80% with max pooling.

8.5 How do feature maps represent image characteristics?

The feature maps show that each filter picks up different parts of the input image. Some maps highlight the brighter areas and edges, while others show different patterns and details. These features are then passed on to the next layers, which use them for the final classification.

8.6 Why do CNNs require fewer parameters than MLPs?

In our experiment, the CNN had 268,650 trainable parameters while working with 32×32 RGB images. This is because the same convolution filters are used at different positions in the image instead of creating separate weights for every pixel. This helps keep the number of parameters lower compared to directly connecting every pixel to all the neurons.

9 Conclusion

In this experiment, I implemented a CNN to classify the CIFAR-10 images and checked how different settings affected its performance. I tried different kernel sizes, strides and padding to see how they changed the feature-map size. I also compared max pooling with average pooling, different numbers of filters, and the ReLU and Sigmoid activation functions.

The final CNN achieved a training accuracy of 91.92% and a testing accuracy of 63.52%. From the comparisons, average pooling gave a slightly better result than max pooling, while increasing the number of filters improved the accuracy only by a small amount. ReLU also performed better than Sigmoid in our experiment. From the training graphs and confusion matrix, I could see that the model learned the training data well, but some overfitting was present. The model also had more difficulty classifying similar classes, especially Cat and Dog.

10 References

1. Goodfellow, I., Bengio, Y., and Courville, A., *Deep Learning*, MIT Press.
2. Bishop, C. M., *Pattern Recognition and Machine Learning*, Springer.
3. Haykin, S., *Neural Networks and Learning Machines*, Pearson.
4. TensorFlow Documentation.

5. CIFAR-10 Dataset Documentation.